

Freestanding Library: Easy [utilities]

Document number: P1642R0

Date: 2019-06-02

Reply-to: Ben Craig <ben dot craig at gmail dot com>

Audience: SG14, Library Evolution Working Group

Change history

R0

Branching from P0829R4. This "omnibus" paper is still the direction I am aiming for. However, it is too difficult to review. It needs to change with almost every meeting. Therefore, it is getting split up into smaller, more manageable chunks.

Limiting paper to the [utilities] clause.

Introduction

This paper proposes adding many of the facilities in the [utilities] clause to the freestanding subset of C++. The paper will only be adding complete entities, and will not tackle partial classes. In other words, classes like `pair` and `tuple` are being added, but trickier classes like `optional`, `variant`, and `bitset` will come in another paper.

Motivation and Design

Many existing facilities in the C++ standard library could be used without trouble in freestanding environments. This series of papers will specify the maximal subset of the C++ standard library that does not require an OS or space overhead.

For a more in depth rationale, see P0829.

`<optional>`, `<variant>`, and `<bitset>` are not in this paper, as all have non-essential functions that can throw an exception. `<charconv>` is not in this paper as it will require us to deal with the thorny issue of overload sets involving floating point and non-floating point types. I plan on addressing all four of these headers in later papers, so that the topics in question can be debated in relative isolation.

Wording

The following changes are relative to N4810 from the Post-Kona 2019 mailing, and assume that P1641R0 "Freestanding Library: Rewording the Status Quo" has been applied.

Instructions to the editor:

Please add the following headers to "Table 22 - C++ headers for freestanding implementations":

- `<utility>`
- `<tuple>`
- `<memory>`
- `<functional>`
- `<ratio>`

Instructions to the editor:

Please add a *// freestanding* comment to the beginning of the following synopses. These headers are entirely freestanding.

- `[utility.syn]`
- `[tuple.syn]`
- `[ratio.syn]`

Instructions to the editor:

Move the following macros from "Table ??37?? - Hosted standard library feature-test macros" to "Table ??36?? - Freestanding and hosted standard library feature-test macros"

- `__cpp_lib_addressof_constexpr`
- `__cpp_lib_allocator_traits_is_always_equal`
- `__cpp_lib_apply`
- `__cpp_lib_as_const`
- `__cpp_lib_bind_front`
- `__cpp_lib_boyer_moore_searcher`
- `__cpp_lib_constexpr_misc`
- `__cpp_lib_exchange_function`

- `__cpp_lib_integer_sequence`
- `__cpp_lib_invoke`
- `__cpp_lib_make_from_tuple`
- `__cpp_lib_make_reverse_iterator`
- `__cpp_lib_nonmember_container_access`
- `__cpp_lib_not_fn`
- `__cpp_lib_null_iterators`
- `__cpp_lib_raw_memory_algorithms`
- `__cpp_lib_transparent_operators`
- `__cpp_lib_tuple_element_t`
- `__cpp_lib_tuples_by_type`

Change in [memory.syn]:

???

Header <memory> synopsis

[memory.syn]

- 1 The header <memory> defines several types and function templates that describe properties of pointers and pointer-like types, manage memory for containers and other template types, destroy objects, and construct multiple objects in uninitialized memory buffers ([pointer.traits]-[specialized.algorithms]). The header also defines the templates `unique_ptr`, `shared_ptr`, `weak_ptr`, and various function templates that operate on objects of these types ([smartptr]).

```
namespace std {
    // [pointer.traits], pointer traits
    template<class Ptr> struct pointer_traits; // freestanding
    template<class T> struct pointer_traits<T*>; // freestanding

    // [pointer.conversion], pointer conversion
    template<class Ptr>
        auto to_address(const Ptr& p) noexcept; // freestanding
    template<class T>
        constexpr T* to_address(T* p) noexcept; // freestanding

    // [util.dynamic.safety], pointer safety
    enum class pointer_safety { relaxed, preferred, strict };
    void declare_reachable(void* p);
    template<class T>
        T* undeclare_reachable(T* p);
    void declare_no_pointers(char* p, size_t n);
    void undeclare_no_pointers(char* p, size_t n);
    pointer_safety get_pointer_safety() noexcept;

    // [ptr.align], pointer alignment
    void* align(size_t alignment, size_t size, void*& ptr, size_t& space); // freestanding
    template<size_t N, class T>
        [[nodiscard]] constexpr T* assume_aligned(T* ptr); // freestanding

    // [allocator.tag], allocator argument tag
    struct allocator_arg_t { explicit allocator_arg_t() = default; }; // freestanding
    inline constexpr allocator_arg_t allocator_arg{}; // freestanding

    // [allocator.uses], uses_allocator
    template<class T, class Alloc> struct uses_allocator; // freestanding

    // [allocator.uses.trait], uses_allocator
    template<class T, class Alloc>
        inline constexpr bool uses_allocator_v = uses_allocator<T, Alloc>::value; // freestanding

    // [allocator.uses.construction], uses-allocator construction
    template <class T, class Alloc, class... Args>
        auto uses_allocator_construction_args(const Alloc& alloc, Args&&... args) -> see below;
    template <class T, class Alloc, class Tuple1, class Tuple2>
        auto uses_allocator_construction_args(const Alloc& alloc, piecewise_construct_t,
            Tuple1&& x, Tuple2&& y) -> see below;

    template <class T, class Alloc>
        auto uses_allocator_construction_args(const Alloc& alloc) -> see below;
    template <class T, class Alloc, class U, class V>
        auto uses_allocator_construction_args(const Alloc& alloc, U&& u, V&& v) -> see below;
    template <class T, class Alloc, class U, class V>
        auto uses_allocator_construction_args(const Alloc& alloc, const pair<U,V>& pr) -> see below;
    template <class T, class Alloc, class U, class V>
        auto uses_allocator_construction_args(const Alloc& alloc, pair<U,V>&& pr) -> see below;
    template <class T, class Alloc, class... Args>
        T make_obj_using_allocator(const Alloc& alloc, Args&&... args);
```

```

template <class T, class Alloc, class... Args>
    T* uninitialized_construct_using_allocator(T* p, const Alloc& alloc, Args&&... args);

// [allocator.traits], allocator traits
template<class Alloc> struct allocator_traits; // freestanding

// [default.allocator], the default allocator
template<class T> class allocator;
template<class T, class U>
    bool operator==(const allocator<T>&, const allocator<U>&) noexcept;
template<class T, class U>
    bool operator!=(const allocator<T>&, const allocator<U>&) noexcept;

// [specialized.algorithms], specialized algorithms
// [special.mem.concepts], special memory concepts
template<class I>
    concept no-throw-input-iterator = see below; // exposition only
template<class I>
    concept no-throw-forward-iterator = see below; // exposition only
template<class S, class I>
    concept no-throw-sentinel = see below; // exposition only
template<class R>
    concept no-throw-input-range = see below; // exposition only
template<class R>
    concept no-throw-forward-range = see below; // exposition only

template<class T>
    constexpr T* addressof(T& r) noexcept; // freestanding
template<class T>
    const T* addressof(const T&) = delete; // freestanding
template<class ForwardIterator>
    void uninitialized_default_construct(ForwardIterator first, ForwardIterator last); // freestanding
template<class ExecutionPolicy, class ForwardIterator>
    void uninitialized_default_construct(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                                        ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Size>
    ForwardIterator uninitialized_default_construct_n(ForwardIterator first, Size n); // freestanding
template<class ExecutionPolicy, class ForwardIterator, class Size>
    ForwardIterator uninitialized_default_construct_n(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                                                    ForwardIterator first, Size n);

namespace ranges {
    template<no-throw-forward-iterator I, no-throw-sentinel<I> S>
        requires DefaultConstructible<iter_value_t<I>>
        I uninitialized_default_construct(I first, S last); // freestanding
    template<no-throw-forward-range R>
        requires DefaultConstructible<iter_value_t<iterator_t<R>>>
        safe_iterator_t<R> uninitialized_default_construct(R&& r); // freestanding

    template<no-throw-forward-iterator I>
        requires DefaultConstructible<iter_value_t<I>>
        I uninitialized_default_construct_n(I first, iter_difference_t<I> n); // freestanding
}

template<class ForwardIterator>
    void uninitialized_value_construct(ForwardIterator first, ForwardIterator last); // freestanding
template<class ExecutionPolicy, class ForwardIterator>
    void uninitialized_value_construct(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                                      ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Size>
    ForwardIterator uninitialized_value_construct_n(ForwardIterator first, Size n); // freestanding
template<class ExecutionPolicy, class ForwardIterator, class Size>
    ForwardIterator uninitialized_value_construct_n(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                                                  ForwardIterator first, Size n);

namespace ranges {
    template<no-throw-forward-iterator I, no-throw-sentinel<I> S>
        requires DefaultConstructible<iter_value_t<I>>
        I uninitialized_value_construct(I first, S last); // freestanding
    template<no-throw-forward-range R>
        requires DefaultConstructible<iter_value_t<iterator_t<R>>>
        safe_iterator_t<R> uninitialized_value_construct(R&& r); // freestanding

    template<no-throw-forward-iterator I>
        requires DefaultConstructible<iter_value_t<I>>

```

```

        I uninitialized_value_construct_n(I first, iter_difference_t<I> n); // freestanding
    }

    template<class InputIterator, class ForwardIterator>
        ForwardIterator uninitialized_copy(InputIterator first, InputIterator last,
                                          ForwardIterator result); // freestanding
    template<class ExecutionPolicy, class InputIterator, class ForwardIterator>
        ForwardIterator uninitialized_copy(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                                          InputIterator first, InputIterator last,
                                          ForwardIterator result);
    template<class InputIterator, class Size, class ForwardIterator>
        ForwardIterator uninitialized_copy_n(InputIterator first, Size n,
                                          ForwardIterator result); // freestanding
    template<class ExecutionPolicy, class InputIterator, class Size, class ForwardIterator>
        ForwardIterator uninitialized_copy_n(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                                          InputIterator first, Size n,
                                          ForwardIterator result);

    namespace ranges {
        template<class I, class O>
            using uninitialized_copy_result = copy_result<I, O>; // freestanding
        template<InputIterator I, Sentinel<I> S1,
                 no-throw-forward-iterator O, no-throw-sentinel<O> S2>
            requires Constructible<iter_value_t<O>, iter_reference_t<I>>
                uninitialized_copy_result<I, O>
                uninitialized_copy(I ifirst, S1 ilast, O ofirst, S2 olast); // freestanding
        template<InputRange IR, no-throw-forward-range OR>
            requires Constructible<iter_value_t<iterator_t<OR>>, iter_reference_t<iterator_t<IR>>>
                uninitialized_copy_result<safe_iterator_t<IR>, safe_iterator_t<OR>>
                uninitialized_copy(IR&& input_range, OR&& output_range); // freestanding

        template<class I, class O>
            using uninitialized_copy_n_result = uninitialized_copy_result<I, O>; // freestanding
        template<InputIterator I, no-throw-forward-iterator O, no-throw-sentinel<O> S>
            requires Constructible<iter_value_t<O>, iter_reference_t<I>>
                uninitialized_copy_n_result<I, O>
                uninitialized_copy_n(I ifirst, iter_difference_t<I> n, O ofirst, S olast); // freestanding
    }

    template<class InputIterator, class ForwardIterator>
        ForwardIterator uninitialized_move(InputIterator first, InputIterator last,
                                          ForwardIterator result); // freestanding
    template<class ExecutionPolicy, class InputIterator, class ForwardIterator>
        ForwardIterator uninitialized_move(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                                          InputIterator first, InputIterator last,
                                          ForwardIterator result);
    template<class InputIterator, class Size, class ForwardIterator>
        pair<InputIterator, ForwardIterator> uninitialized_move_n(InputIterator first, Size n,
                                                                ForwardIterator result); // freestanding
    template<class ExecutionPolicy, class InputIterator, class Size, class ForwardIterator>
        pair<InputIterator, ForwardIterator> uninitialized_move_n(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                                                                InputIterator first, Size n,
                                                                ForwardIterator result);

    namespace ranges {
        template<class I, class O>
            using uninitialized_move_result = uninitialized_copy_result<I, O>; // freestanding
        template<InputIterator I, Sentinel<I> S1,
                 no-throw-forward-iterator O, no-throw-sentinel<O> S2>
            requires Constructible<iter_value_t<O>, iter_rvalue_reference_t<I>>
                uninitialized_move_result<I, O>
                uninitialized_move(I ifirst, S1 ilast, O ofirst, S2 olast); // freestanding
        template<InputRange IR, no-throw-forward-range OR>
            requires Constructible<iter_value_t<iterator_t<OR>>,
                                   iter_rvalue_reference_t<iterator_t<IR>>>
                uninitialized_move_result<safe_iterator_t<IR>, safe_iterator_t<OR>>
                uninitialized_move(IR&& input_range, OR&& output_range); // freestanding

        template<class I, class O>
            using uninitialized_move_n_result = uninitialized_copy_result<I, O>; // freestanding
        template<InputIterator I,
                 no-throw-forward-iterator O, no-throw-sentinel<O> S>
            requires Constructible<iter_value_t<O>, iter_rvalue_reference_t<I>>
                uninitialized_move_n_result<I, O>
                uninitialized_move_n(I ifirst, iter_difference_t<I> n, O ofirst, S olast); // freestanding
    }

```

```

template<class ForwardIterator, class T>
    void uninitialized_fill(ForwardIterator first, ForwardIterator last, const T& x); // freestanding
template<class ExecutionPolicy, class ForwardIterator, class T>
    void uninitialized_fill(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                           ForwardIterator first, ForwardIterator last, const T& x);
template<class ForwardIterator, class Size, class T>
    ForwardIterator uninitialized_fill_n(ForwardIterator first, Size n, const T& x); // freestanding
template<class ExecutionPolicy, class ForwardIterator, class Size, class T>
    ForwardIterator uninitialized_fill_n(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                                       ForwardIterator first, Size n, const T& x);

namespace ranges {
    template<no-throw-forward-iterator I, no-throw-sentinel<I> S, class T>
        requires Constructible<iter_value_t<I>, const T&>
        I uninitialized_fill(I first, S last, const T& x); // freestanding
    template<no-throw-forward-range R, class T>
        requires Constructible<iter_value_t<iterator_t<R>>, const T&>
        safe_iterator_t<R> uninitialized_fill(R&& r, const T& x); // freestanding

    template<no-throw-forward-iterator I, class T>
        requires Constructible<iter_value_t<I>, const T&>
        I uninitialized_fill_n(I first, iter_difference_t<I> n, const T& x); // freestanding
}

template<class T>
    void destroy_at(T* location); // freestanding
template<class ForwardIterator>
    void destroy(ForwardIterator first, ForwardIterator last); // freestanding
template<class ExecutionPolicy, class ForwardIterator>
    void destroy(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
               ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Size>
    ForwardIterator destroy_n(ForwardIterator first, Size n); // freestanding
template<class ExecutionPolicy, class ForwardIterator, class Size>
    ForwardIterator destroy_n(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                             ForwardIterator first, Size n);

namespace ranges {
    template<Destructible T>
        void destroy_at(T* location) noexcept; // freestanding

    template<no-throw-input-iterator I, no-throw-sentinel<I> S>
        requires Destructible<iter_value_t<I>>
        I destroy(I first, S last) noexcept; // freestanding
    template<no-throw-input-range R>
        requires Destructible<iter_value_t<iterator_t<R>>
        safe_iterator_t<R> destroy(R&& r) noexcept; // freestanding

    template<no-throw-input-iterator I>
        requires Destructible<iter_value_t<I>>
        I destroy_n(I first, iter_difference_t<I> n) noexcept; // freestanding
}

// [unique.ptr], class template unique_ptr
template<class T> struct default_delete; // freestanding
template<class T> struct default_delete<T[]>; // freestanding
template<class T, class D = default_delete<T>> class unique_ptr; // freestanding
template<class T, class D> class unique_ptr<T[], D>; // freestanding

template<class T, class... Args>
    unique_ptr<T> make_unique(Args&&... args); // T is not array
template<class T>
    unique_ptr<T> make_unique(size_t n); // T is U[]
template<class T, class... Args>
    unspecified make_unique(Args&&...) = delete; // T is U[N]

template<class T>
    unique_ptr<T> make_unique_default_init(); // T is not array
template<class T>
    unique_ptr<T> make_unique_default_init(size_t n); // T is U[]
template<class T, class... Args>
    unspecified make_unique_default_init(Args&&...) = delete; // T is U[N]

template<class T, class D> // freestanding

```

```

void swap(unique_ptr<T, D>& x, unique_ptr<T, D>& y) noexcept;

template<class T1, class D1, class T2, class D2> // freestanding
    bool operator==(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2> // freestanding
    bool operator!=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2> // freestanding
    bool operator<(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2> // freestanding
    bool operator>(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2> // freestanding
    bool operator<=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2> // freestanding
    bool operator>=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);

template<class T, class D> // freestanding
    bool operator==(const unique_ptr<T, D>& x, nullptr_t) noexcept;
template<class T, class D> // freestanding
    bool operator==(nullptr_t, const unique_ptr<T, D>& y) noexcept;
template<class T, class D> // freestanding
    bool operator!=(const unique_ptr<T, D>& x, nullptr_t) noexcept;
template<class T, class D> // freestanding
    bool operator!=(nullptr_t, const unique_ptr<T, D>& y) noexcept;
template<class T, class D> // freestanding
    bool operator<(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D> // freestanding
    bool operator<(nullptr_t, const unique_ptr<T, D>& y);
template<class T, class D> // freestanding
    bool operator>(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D> // freestanding
    bool operator>(nullptr_t, const unique_ptr<T, D>& y);
template<class T, class D> // freestanding
    bool operator<=(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D> // freestanding
    bool operator<=(nullptr_t, const unique_ptr<T, D>& y);
template<class T, class D> // freestanding
    bool operator>=(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D> // freestanding
    bool operator>=(nullptr_t, const unique_ptr<T, D>& y);

template<class E, class T, class Y, class D>
    basic_ostream<E, T>& operator<<(basic_ostream<E, T>& os, const unique_ptr<Y, D>& p);

// [util.smartptr.weak.bad], class bad_weak_ptr
class bad_weak_ptr;

// [util.smartptr.shared], class template shared_ptr
template<class T> class shared_ptr;

// [util.smartptr.shared.create], shared_ptr creation
template<class T, class... Args>
    shared_ptr<T> make_shared(Args&&... args); // T is not array
template<class T, class A, class... Args>
    shared_ptr<T> allocate_shared(const A& a, Args&&... args); // T is not array

template<class T>
    shared_ptr<T> make_shared(size_t N); // T is U[]
template<class T, class A>
    shared_ptr<T> allocate_shared(const A& a, size_t N); // T is U[]

template<class T>
    shared_ptr<T> make_shared(); // T is U[N]
template<class T, class A>
    shared_ptr<T> allocate_shared(const A& a); // T is U[N]

template<class T>
    shared_ptr<T> make_shared(size_t N, const remove_extent_t<T>& u); // T is U[]
template<class T, class A>
    shared_ptr<T> allocate_shared(const A& a, size_t N,
                                const remove_extent_t<T>& u); // T is U[]

template<class T>
    shared_ptr<T> make_shared(const remove_extent_t<T>& u); // T is U[N]
template<class T, class A>
    shared_ptr<T> allocate_shared(const A& a, const remove_extent_t<T>& u); // T is U[N]

```

```

template<class T>
    shared_ptr<T> make_shared_default_init(); // T is not U[]
template<class T, class A>
    shared_ptr<T> allocate_shared_default_init(const A& a); // T is not U[]

template<class T>
    shared_ptr<T> make_shared_default_init(size_t N); // T is U[]
template<class T, class A>
    shared_ptr<T> allocate_shared_default_init(const A& a, size_t N); // T is U[]

// [util.smartptr.shared.cmp], shared_ptr comparisons
template<class T, class U>
    bool operator==(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    bool operator!=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    bool operator<(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    bool operator>(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    bool operator<=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    bool operator>=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;

template<class T>
    bool operator==(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
    bool operator==(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
    bool operator!=(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
    bool operator!=(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
    bool operator<(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
    bool operator<(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
    bool operator<=(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
    bool operator<=(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
    bool operator>(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
    bool operator>(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
    bool operator>=(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
    bool operator>=(nullptr_t, const shared_ptr<T>& y) noexcept;

// [util.smartptr.shared.spec], shared_ptr specialized algorithms
template<class T>
    void swap(shared_ptr<T>& a, shared_ptr<T>& b) noexcept;

// [util.smartptr.shared.cast], shared_ptr casts
template<class T, class U>
    shared_ptr<T> static_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    shared_ptr<T> static_pointer_cast(shared_ptr<U>&& r) noexcept;
template<class T, class U>
    shared_ptr<T> dynamic_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    shared_ptr<T> dynamic_pointer_cast(shared_ptr<U>&& r) noexcept;
template<class T, class U>
    shared_ptr<T> const_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    shared_ptr<T> const_pointer_cast(shared_ptr<U>&& r) noexcept;
template<class T, class U>
    shared_ptr<T> reinterpret_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    shared_ptr<T> reinterpret_pointer_cast(shared_ptr<U>&& r) noexcept;

// [util.smartptr.getdeleter], shared_ptr get_deleter
template<class D, class T>
    D* get_deleter(const shared_ptr<T>& p) noexcept;

// [util.smartptr.shared.io], shared_ptr I/O

```



```

template<class E, class T, class Y>
    basic_ostream<E, T>& operator<<(basic_ostream<E, T>& os, const shared_ptr<Y>& p);

// [util.smartptr.weak], class template weak_ptr
template<class T> class weak_ptr;

// [util.smartptr.weak.spec], weak_ptr specialized algorithms
template<class T> void swap(weak_ptr<T>& a, weak_ptr<T>& b) noexcept;

// [util.smartptr.ownerless], class template owner_less
template<class T = void> struct owner_less;

// [util.smartptr.enab], class template enable_shared_from_this
template<class T> class enable_shared_from_this;

// [util.smartptr.hash], hash support
template<class T> struct hash; // freestanding
template<class T, class D> struct hash<unique_ptr<T, D>>; // freestanding
template<class T> struct hash<shared_ptr<T>>;

// [util.smartptr.atomic], atomic smart pointers
template<class T> struct atomic; // freestanding
template<class T> struct atomic<shared_ptr<T>>;
template<class T> struct atomic<weak_ptr<T>>;
}

```

Change in [functional.syn]:

???

Header <functional> synopsis

[functional.syn]

```

namespace std {
    // [func.invoke], invoke
    template<class F, class... Args>
        invoke_result_t<F, Args...> invoke(F&& f, Args&&... args)
            noexcept(is_nothrow_invocable_v<F, Args...>); // freestanding

    // [refwrap], reference_wrapper
    template<class T> class reference_wrapper; // freestanding

    template<class T> reference_wrapper<T> ref(T&) noexcept; // freestanding
    template<class T> reference_wrapper<const T> cref(const T&) noexcept; // freestanding
    template<class T> void ref(const T&&) = delete; // freestanding
    template<class T> void cref(const T&&) = delete; // freestanding

    template<class T> reference_wrapper<T> ref(reference_wrapper<T>) noexcept; // freestanding
    template<class T> reference_wrapper<const T> cref(reference_wrapper<T>) noexcept; // freestanding

    template<class T> struct unwrap_reference; // freestanding
    template<class T> struct unwrap_ref_decay : unwrap_reference<decay_t<T>> {}; // freestanding
    template<class T> using unwrap_ref_decay_t = typename unwrap_ref_decay<T>::type; // freestanding

    // [arithmetic.operations], arithmetic operations
    template<class T = void> struct plus; // freestanding
    template<class T = void> struct minus; // freestanding
    template<class T = void> struct multiplies; // freestanding
    template<class T = void> struct divides; // freestanding
    template<class T = void> struct modulus; // freestanding
    template<class T = void> struct negate; // freestanding
    template<> struct plus<void>; // freestanding
    template<> struct minus<void>; // freestanding
    template<> struct multiplies<void>; // freestanding
    template<> struct divides<void>; // freestanding
    template<> struct modulus<void>; // freestanding
    template<> struct negate<void>; // freestanding

    // [comparisons], comparisons
    template<class T = void> struct equal_to; // freestanding
    template<class T = void> struct not_equal_to; // freestanding
    template<class T = void> struct greater; // freestanding
    template<class T = void> struct less; // freestanding

```



```

template<class T = void> struct greater_equal; // freestanding
template<class T = void> struct less_equal; // freestanding
template<> struct equal_to<void>; // freestanding
template<> struct not_equal_to<void>; // freestanding
template<> struct greater<void>; // freestanding
template<> struct less<void>; // freestanding
template<> struct greater_equal<void>; // freestanding
template<> struct less_equal<void>; // freestanding

// [logical.operations], logical operations
template<class T = void> struct logical_and; // freestanding
template<class T = void> struct logical_or; // freestanding
template<class T = void> struct logical_not; // freestanding
template<> struct logical_and<void>; // freestanding
template<> struct logical_or<void>; // freestanding
template<> struct logical_not<void>; // freestanding

// [bitwise.operations], bitwise operations
template<class T = void> struct bit_and; // freestanding
template<class T = void> struct bit_or; // freestanding
template<class T = void> struct bit_xor; // freestanding
template<class T = void> struct bit_not; // freestanding
template<> struct bit_and<void>; // freestanding
template<> struct bit_or<void>; // freestanding
template<> struct bit_xor<void>; // freestanding
template<> struct bit_not<void>; // freestanding

// [func.identity], identity
struct identity; // freestanding

// [func.not_fn], function template not_fn
template<class F> unspecified not_fn(F&& f); // freestanding

// [func.bind_front], function template bind_front
template <class F, class... Args> unspecified bind_front(F&&, Args&&...); // freestanding

// [func.bind], bind
template<class T> struct is_bind_expression; // freestanding
template<class T> struct is_placeholder; // freestanding

template<class F, class... BoundArgs>
    unspecified bind(F&&, BoundArgs&&...); // freestanding
template<class R, class F, class... BoundArgs>
    unspecified bind(F&&, BoundArgs&&...); // freestanding

namespace placeholders {
    // M is the implementation-defined number of placeholders
    see below _1; // freestanding
    see below _2; // freestanding
    .
    .
    .
    see below _M; // freestanding
}

// [func.memfn], member function adaptors
template<class R, class T>
    unspecified mem_fn(R T::*) noexcept; // freestanding

// [func.wrap], polymorphic function wrappers
class bad_function_call;

template<class> class function; // not defined
template<class R, class... ArgTypes> class function<R(ArgTypes...)>;

template<class R, class... ArgTypes>
    void swap(function<R(ArgTypes...)>&, function<R(ArgTypes...)>&) noexcept;

template<class R, class... ArgTypes>
    bool operator==(const function<R(ArgTypes...)>&, nullptr_t) noexcept;
template<class R, class... ArgTypes>
    bool operator==(nullptr_t, const function<R(ArgTypes...)>&) noexcept;
template<class R, class... ArgTypes>

```

```

    bool operator!=(const function<R(ArgTypes...)>&, nullptr_t) noexcept;
template<class R, class... ArgTypes>
    bool operator!=(nullptr_t, const function<R(ArgTypes...)>&) noexcept;

// [func.search], searchers
template<class ForwardIterator, class BinaryPredicate = equal_to<>>
    class default_searcher; // freestanding

template<class RandomAccessIterator,
        class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
        class BinaryPredicate = equal_to<>>
    class boyer_moore_searcher;

template<class RandomAccessIterator,
        class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
        class BinaryPredicate = equal_to<>>
    class boyer_moore_horspool_searcher;

// [unord.hash], hash function primary template
template<class T>
    struct hash; // freestanding

// [func.bind], function object binders
template<class T>
    inline constexpr bool is_bind_expression_v = is_bind_expression<T>::value; // freestanding
template<class T>
    inline constexpr int is_placeholder_v = is_placeholder<T>::value; // freestanding

namespace ranges {
    // [range.cmp], concept-constrained comparisons
    struct equal_to; // freestanding
    struct not_equal_to; // freestanding
    struct greater; // freestanding
    struct less; // freestanding
    struct greater_equal; // freestanding
    struct less_equal; // freestanding
}
}

```

Acknowledgements

Thanks to Brandon Streiff, Joshua Cannon, Phil Hindman, and Irwan Djajadi for reviewing P0829.

Thanks to Odin Holmes for providing feedback and helping publicize P0829.

Thanks to Paul Bendixen for providing feedback while prototyping P0829.

Similar work was done in the C++11 timeframe by Lawrence Cowl and Alberto Ganesh Barbati in [N3256](#).